

API for PC Sampling on Various Devices via PAPI

Working Group Outbrief

Scalable Tools Workshop
July 26-30, 2026

Motivation

- Various GPU vendors expose PC sampling:
 - NVIDIA, AMD, Intel
- Tools utilize PC sampling
 - HPCToolkit
- Some tools may want to use it, but not have to re-instrument for every vendor API.
- PAPI serves as a middleware for various tools.

PC Sample Format and Examples

From rocm-systems/projects/rocprofiler-sdk/samples:

```
(code_obj_id, offset): (8, 0x2b08), timestamp:  
8972646575471755, exec: 1, workgroup_id (x= 276, y= 380,  
z= 0), wave_in_group: 0, chiplet: 1, dispatch_id: 711,  
correlation: {internal= 711, external= 0}, wave issued  
ROCPROFILER_PC_SAMPLING_INSTRUCTION_TYPE_SCALAR instruction,  
two VALU instructions issued: 0, arbiter state:  
{pipe issued: (VALU: 0, MATRIX: 0, LDS: 0, LDS_DIRECT: 0,  
SCALAR: 1, TEX: 0, FLAT: 0, EXPORT: 0, MISC: 0),  
  pipe stalled: (VALU: 0, MATRIX: 0, LDS: 0, LDS_DIRECT: 0,  
SCALAR: 0, TEX: 0, FLAT: 0, EXPORT: 0, MISC: 0)}
```

PC Sample as a Struct

There are many fields...

- ID of code obj containing PC
- Offset within the code obj
- Timestamp of sample
- Dispatch ID
- Workgroup ID (X, Y, Z)
- Workgroup wave index
- Shader engine ID
- Compute unit ID
- SIMD ID
- Instruction type
- Stall reason
- Execution mask
- Logical GPU device ID
- Wave issued flag
- Chiplet ID
- ...

Unifying the PC Sample Structure

- HPCToolkit team has created a model for this across vendors.

Further taxonomy:

- ROCPROFILER_PC_SAMPLING_RECORD_HOST_TRAP_V0_SAMPLE
- ROCPROFILER_PC_SAMPLING_RECORD_STOCHASTIC_V0_SAMPLE
- Do we return a struct that always has the same fields? Use zero-values for the fields not on that arch?

How is PC Sampling Used in Tools?

- How many samples do you want at a time? Does the user give a number *a priori*?
- Which “offsets” were the most frequently sampled?
- How to access samples? Index by offset?

Ideas from Discussion Group

- Aggregate/tally the boolean 0's and 1's between start and stop.
- Rocprofiler-sdk/cxx/include/code_object headers allow you to get the code object back from the (object ID, offset) pair.
- Sass-to-source correlation in NVIDIA tools.
- PC samples are global w.r.t. which GCDs for which it is active. It's turned on *everywhere* for *everything* until you turn it back off.
- How would a user set the sampling rate?
 - Some mechanism could set a (key, value) pair, where the key is “sampling rate” and the value is the rate value itself.

What about AMD SPM?

- This is also a large volume of records.
- PAPI's rocp_sdk component has “Sampling” and “Dispatch” modes to leverage the resources given by AMD.
- Create a new mode “SPM” which exposes statistical quartiles? Standard deviation of the counter between start and stop.

Further Considerations: Thread-Level Tracing

- PC samples have thread ID field in the record.
- External correlation ID is another path to attribution.
- ATT (Advanced Thread Tracing): is most useful for small kernels, and sequences of small kernels. Can detect thread divergence. Near cycle-accurate instruction tracing.
 - <https://github.com/rocm/rocprof-compute-viewer> shows what thread-level tracing does.

Participants

- John Mellor-Crummey
- David Boehme
- Giovanni Baraldi
- Jonathan Madsen
- Chris Ashton
- Zhiqiang Ma
- Anthony Danalis
- Treece Burgess
- Daniel Barry